IH

InternHub

Intern & Project Tracker — Project Report

A full-stack web application where mentors manage interns, assign projects and score submissions. Built as a training project demonstrating **database integration** (schema design, CRUD, constraints, secure queries) and **frontend–backend integration** (async fetch, dynamic UI, robust error handling).

Node.js + Express 5 | PostgreSQL 16 | pg · raw parameterized SQL | express-validator | Vanilla HTML / CSS / JS | REST · JSON

DOCUMENT
Technical project report

DATE
24 October 2026

VERIFICATION
55 API checks · 54 browser checks · all passing

Contents

1 Executive summary

2 Objectives and how they are met

3 System architecture

4 Technology stack and project structure

5 Database design

6 Data integrity and security

7 REST API reference

8 Frontend design and integration

9 Application walkthrough (screenshots)

10 Testing and verification

11 Setup and running

12 Design decisions, limitations and future work

1 Executive summary

InternHub lets a mentor see every intern at a glance, filter and search them, open an intern's profile, assign projects and score submitted work — each score checked against the project's maximum.

5	21	3	109
database tables	REST endpoints	frontend pages	automated checks passed

The backend is an Express API that talks to PostgreSQL using the `pg` driver and hand-written, fully parameterized SQL — no ORM. All input is validated, PostgreSQL constraint violations are translated into meaningful HTTP status codes (409 / 422), and every error follows a single JSON shape. The frontend is plain HTML, CSS and JavaScript (ES modules, no build step) served from a separate origin; it talks to the API through one central `request()` wrapper and renders all data with `createElement` / `textContent` to prevent XSS.

Every data view has explicit **loading**, **empty** and **error-with-Retry** states, so the UI never shows a blank screen — even with the backend stopped. Every endpoint and failure case was exercised with `curl`, and every user flow was clicked through in headless Chrome.

> **Outcome**
>
> All required features are implemented and verified: full CRUD, filtering, debounced search and pagination, field-level validation errors, custom confirm dialogs, SQL-injection and XSS resistance, CORS restricted to the frontend origin, and graceful degradation when the API is unavailable.

2 Objectives and how they are met

The brief defines two learning phases. The tables below map each objective to the concrete implementation.

Phase 1 — Database integration

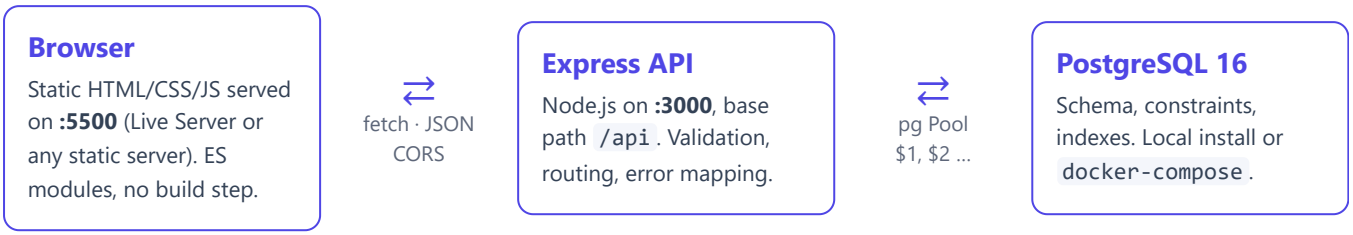
OBJECTIVE	IMPLEMENTATION
Schema design	Five normalized tables (`mentors`, `interns`, `intern_profiles`, `projects`, `submissions`) expressing all three relationship types: 1:Many, 1:1 and Many:Many. Indexes on every foreign key used in JOINs and filters.
CRUD	Full create / read / update / delete for interns (including both `PUT` full replace and `PATCH` partial update), mentors and projects; create / update / delete for submissions; an aggregate stats endpoint using `GROUP BY`.
Constraints & integrity	`NOT NULL`, `UNIQUE`, `CHECK` (track, status, `max_score > 0`, `score >= 0`), foreign keys with `ON DELETE CASCADE` / `SET NULL`, composite `UNIQUE (intern_id, project_id)`. Multi-statement writes run in transactions. The cross-table rule *score ≤ project.max_score* is enforced in the API under row locks.
Secure queries	Every query uses `$1, $2…` placeholders. Dynamic SQL (filters, PATCH, profile upsert) is assembled only from **whitelisted column names**; user values always travel as parameters. LIKE wildcards in search terms are escaped.

Phase 2 — Frontend–backend integration

OBJECTIVE	IMPLEMENTATION
API integration	One `request(path, {method, body})` function in `js/api.js` wraps `fetch`; every page goes through it. CORS on the server allows only the configured frontend origin.
Async requests	`async/await` throughout. The dashboard loads stats, interns, projects and mentors in parallel with `Promise.all`. Debounced search uses `AbortController` so stale responses never overwrite newer ones.
Error handling	Server: one error shape `{ error: { status, message, details? } }`, central middleware, no stack traces leaked. Client: custom `ApiError(status, message, details)`; 422/409 errors are shown beside the offending form field, others as toasts; loading / empty / error+Retry states on every view.
Dynamic UI	All lists are built with `createElement` + `textContent` (never `innerHTML` with data). The UI refreshes affected regions after each write — no full page reloads. Submit buttons are disabled while a request is in flight.
Full-stack flow	Form → client `request()` → Express validation → parameterized SQL → PostgreSQL constraint → mapped HTTP status → field error or refreshed view.

3 System architecture

InternHub is a classic three-tier application. The frontend and the API run on different origins (ports 5500 and 3000), which makes CORS a real, tested part of the design rather than an afterthought.



Request lifecycle (example: creating an intern)

1. **UI** — the user submits the "Add intern" form. `dashboard.js` reads the fields, disables the submit button and calls `api.createIntern(data)`.
2. **Client wrapper** — `request()` sets `Content-Type: application/json`, stringifies the body and calls `fetch`. Because the request carries JSON, the browser first sends a CORS **preflight** (`OPTIONS`), which the `cors` middleware answers for the allowed origin only.
3. **Middleware** — a guard rejects non-JSON bodies (415); `express.json()` parses the body (400 on malformed JSON).
4. **Validation** — `express-validator` chains check types, required fields, email format, track enum, positive-integer ids and URL protocols. Failures become a 422 with per-field `details`.
5. **Database** — inside a transaction the intern row is inserted and, if supplied, the 1:1 profile row is upserted. All values are bound parameters.
6. **Error mapping** — if PostgreSQL raises `23505` (duplicate email) the central error handler returns **409** with `{ field: "email", message: "An intern with this email already exists" }`.
7. **Response** — on success the API returns **201** with the created intern (joined with mentor and profile).
8. **UI update** — the modal closes, a success toast appears, and the interns table, stat cards and projects panel refresh in place. On failure, the message appears directly beneath the offending input.

Backend middleware pipeline

```
cors({ origin: FRONTEND_ORIGIN })   // allow-list; answers OPTIONS preflights
→ requireJson                       // 415 if a POST/PUT/PATCH body is not application/json
→ express.json({ limit: '100kb' }) // 400 malformed JSON, 413 too large
→ routers                           // /api/interns, /mentors, /projects, /submissions, /stats
    → validate([...rules])            // 422 with details[]
    → async handler                   // Express 5 forwards thrown/rejected errors automatically
→ notFound                          // JSON 404 for unknown routes
→ errorHandler                      // HttpError / PG codes → status; logs 5xx server-side only
```

4 Technology stack and project structure

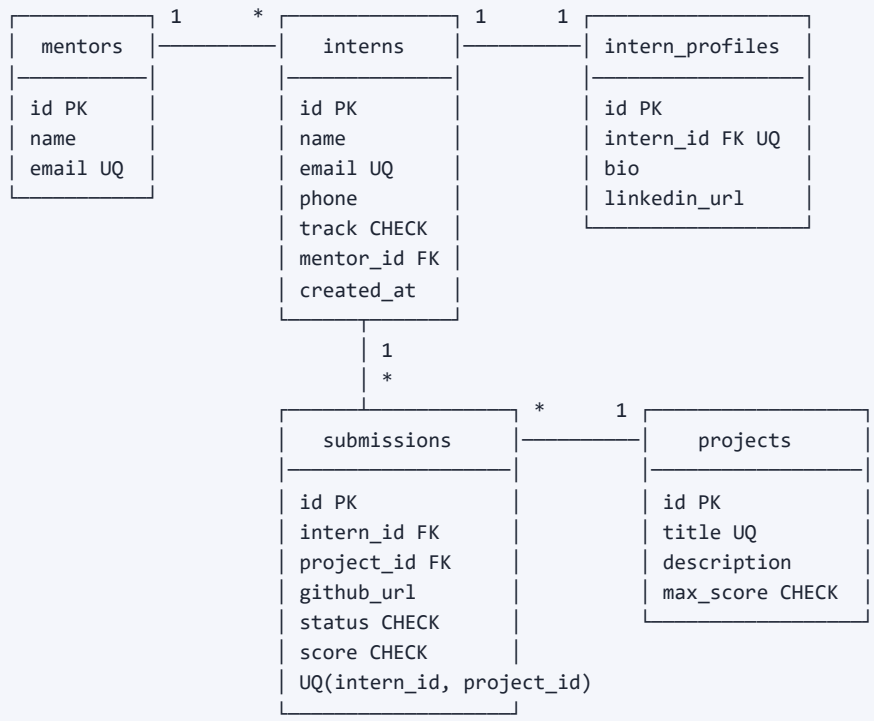
LAYER	TECHNOLOGY	WHY
Runtime	Node.js (LTS), CommonJS	Required by the brief; one module system used consistently on the backend.
HTTP framework	Express 5	Async handlers that throw are forwarded to error middleware automatically — no try/catch boilerplate in routes.
Database	PostgreSQL 16	Strong constraint support, `ON CONFLICT` upserts, `FOR SHARE/UPDATE` row locks, JSON building functions.
DB driver	`pg` (node-postgres)	Raw SQL with native parameter binding; connection pool; explicit transactions.
Validation	`express-validator`	Declarative validation + sanitization chains, per-field error messages.
Config / CORS	`dotenv`, `cors`	`PORT`, `DATABASE_URL`, `FRONTEND_ORIGIN` from `.env`; origin allow-list.
Dev tooling	`nodemon`	Auto-restart during development (`npm run dev`).
Frontend	HTML, CSS, JavaScript (ES modules)	No framework or build step; demonstrates the fundamentals directly.
Infrastructure	`docker-compose.yml` (postgres:16)	One-command database for machines without a local PostgreSQL.

Folder structure

```
internhub/
├── backend/
│   ├── db/
│   │   ├── schema.sql        DROP + CREATE of all tables and indexes (re-runnable)
│   │   ├── seed.sql          3 mentors, 10 interns, 6 profiles, 4 projects, 15 submissions
│   │   ├── pool.js           pg Pool, query(), withTransaction() helper
│   │   └── init.js           runs schema + seed in one transaction  (npm run db:init)
│   ├── routes/               interns.js · mentors.js · projects.js · submissions.js · stats.js
│   ├── middleware/
│   │   ├── validate.js       validate(), idParam(), requireSomeFields(), pickPresent()
│   │   ├── errorHandler.js   HttpError + PostgreSQL error-code mapping, consistent JSON shape
│   │   └── notFound.js       JSON 404 for unknown routes
│   ├── utils/httpError.js    HttpError(status, message, details)
│   ├── server.js             CORS, JSON guard, body parser, routers, 404, error handler
│   ├── .env.example          PORT, DATABASE_URL, FRONTEND_ORIGIN
│   └── package.json          scripts: dev, start, db:init
├── frontend/
│   ├── index.html            dashboard + interns list
│   ├── intern.html           single intern + submissions
│   ├── projects.html         projects CRUD
│   ├── css/style.css         design tokens, components, responsive rules
│   └── js/
│       ├── config.js         API_BASE_URL and shared constants
│       ├── api.js            request() wrapper, ApiError, named API helpers
│       ├── ui.js             el(), view states, toast, confirm dialog, form errors
│       ├── dashboard.js      index.html logic
│       ├── intern.js         intern.html logic
│       └── projects.js       projects.html logic
├── docker-compose.yml
└── README.md
```

5 Database design

Entity-relationship diagram



Relationships

RELATIONSHIP	CARDINALITY	HOW IT IS ENFORCED
mentors → interns	1 : Many	`interns.mentor_id REFERENCES mentors(id) ON DELETE SET NULL` — deleting a mentor keeps their interns, now unassigned.
interns → intern_profiles	1 : 1	`intern_profiles.intern_id` is `UNIQUE NOT NULL` with `ON DELETE CASCADE`; the API upserts with `ON CONFLICT (intern_id)`.
interns ↔ projects	Many : Many	Junction table `submissions` with two cascading FKs and `UNIQUE (intern_id, project_id)` so a project can be assigned to an intern only once.

Table specifications

mentors

`id`	SERIAL PRIMARY KEY
`name`	VARCHAR(100) NOT NULL
`email`	VARCHAR(150) UNIQUE NOT NULL

intern_profiles

`id`	SERIAL PRIMARY KEY
`intern_id`	INT UNIQUE NOT NULL → interns, CASCADE
`bio`	TEXT
`linkedin_url`	TEXT

interns

`id`	SERIAL PRIMARY KEY
`name`	VARCHAR(100) NOT NULL
`email`	VARCHAR(150) UNIQUE NOT NULL
`phone`	VARCHAR(20)
`track`	VARCHAR(20) NOT NULL CHECK IN ('frontend','backend','fullstack')
`mentor_id`	INT → mentors, ON DELETE SET NULL
`created_at`	TIMESTAMP DEFAULT NOW()

projects

`id`	SERIAL PRIMARY KEY
`title`	VARCHAR(150) UNIQUE NOT NULL
`description`	TEXT
`max_score`	INT NOT NULL CHECK (max_score > 0)

submissions

`id`	SERIAL PRIMARY KEY
`intern_id`	INT NOT NULL → interns, CASCADE
`project_id`	INT NOT NULL → projects, CASCADE
`github_url`	TEXT
`status`	VARCHAR(20) NOT NULL DEFAULT 'pending' CHECK IN ('pending','submitted','reviewed')
`score`	INT CHECK (score >= 0)
`UNIQUE (intern_id, project_id)`	

Indexes

```
CREATE INDEX idx_interns_mentor_id      ON interns(mentor_id);       -- mentor JOINs, mentor's intern list
CREATE INDEX idx_submissions_intern_id  ON submissions(intern_id);   -- intern detail, leaderboard GROUP
BY
CREATE INDEX idx_submissions_project_id ON submissions(project_id);  -- project stats, project detail
```

PostgreSQL does not index foreign-key columns automatically; these three cover every JOIN and filter the API performs. `UNIQUE` columns (emails, title, the composite pair) are already backed by implicit unique indexes.

Seed data

`npm run db:init` executes `schema.sql` then `seed.sql` in a single transaction (all-or-nothing, safe to re-run). The seed contains 3 mentors, 10 interns spread across all three tracks (one without a mentor, one with an apostrophe in the name — *Liam O'Brien* — to exercise escaping), 6 profiles, 4 projects with different maximum scores (100, 100, 50, 20) and 15 submissions with a realistic mix of *pending*, *submitted* and *reviewed* statuses. Scores exist only on reviewed work and never exceed the project's maximum.

6 Data integrity and security

6.1 Parameterized queries everywhere

No SQL statement in the codebase is built by concatenating user input. Values are always passed separately to `pg`, which sends them to PostgreSQL as bound parameters — the database never parses them as SQL.

```
// routes/interns.js — dynamic filters: only fixed fragments and $n placeholders are joined
const conditions = [];
const params = [];
if (track) {
  params.push(track);
  conditions.push(`i.track = $${params.length}`);
}
if (search) {
  params.push(`%${escapeLike(search)}%`);           // "50%" matches literally
  conditions.push(`(i.name ILIKE $${params.length} OR i.email ILIKE $${params.length})`);
}
const where = conditions.length ? `WHERE ${conditions.join(' AND ')}` : '';
```

6.2 Whitelisted dynamic updates (PATCH)

Partial updates need a variable column list. The column names come exclusively from a hard-coded whitelist; keys the client sends that are not on the list are ignored.

```
const INTERN_COLUMNS = ['name', 'email', 'phone', 'track', 'mentor_id'];

const fields = pickPresent(req.body, WRITABLE);             // only whitelisted keys survive
const internColumns = INTERN_COLUMNS.filter((c) => Object.hasOwn(fields, c));
const assignments = internColumns.map((c, i) => `${c} = $${i + 2}`);
await client.query(
  `UPDATE interns SET ${assignments.join(', ')} WHERE id = $1`,
  [id, ...internColumns.map((c) => fields[c])]
);
```

6.3 Input validation

RULE	APPLIED TO
Required + type checks	Name, email, track (interns); name, email (mentors); title, max_score (projects); intern_id, project_id (submissions). `null` is rejected for required fields, including in PATCH.
Length limits	Mirror the column sizes (100 / 150 / 20 chars) plus caps on free text (bio 2 000, description 5 000).
Email format	`isEmail()`, trimmed and lower-cased so uniqueness is case-insensitive in practice.
Enums	`track` ∈ frontend/backend/fullstack; `status` ∈ pending/submitted/reviewed.
Positive integer ids	Route params and FK fields: `isInt({ min: 1, max: 2147483647 })` — also prevents INT overflow errors.
URLs	`linkedin_url`, `github_url` must be `http(s)` with protocol, blocking `javascript:` URLs at the source.

RULE	APPLIED TO
Scores	Whole number ≥ 0 and ≤ the project's `max_score` (checked against the database).
Query string	`page` ≥ 1, `limit` 1–100, `search` ≤ 100 chars, `track` enum.
PATCH body	Must contain at least one recognised field, otherwise 422.

6.4 PostgreSQL error mapping

SQLSTATE	MEANING	HTTP	EXAMPLE FIELD-LEVEL MESSAGE
`23505`	unique_violation	**409**	`email`: "An intern with this email already exists"
`23503`	foreign_key_violation	**422**	`mentor_id`: "Selected mentor does not exist"
`23514`	check_violation	**422**	`max_score`: "Max score must be greater than 0"
`22001`	value too long	**422**	"A value is too long"
ECONNREFUSED	database down	**503**	"Database is unavailable, please try again shortly"

Each named constraint (e.g. `interns_email_key`, `submissions_intern_id_project_id_key`) is mapped to the form field it concerns, so the frontend can display the message right next to the right input. Full error objects, stack traces and SQL are logged on the server only; the client receives just `{ error: { status, message, details? } }`.

6.5 Transactions and row locks

- **Intern + profile** writes (POST / PUT / PATCH) happen inside `withTransaction()`: either both rows change or neither does.
- **Scoring a submission** reads the project's `max_score` with `SELECT … FOR SHARE OF p`, then updates. A concurrent attempt to lower the maximum must wait for the commit.
- **Editing a project** locks it with `FOR UPDATE` and refuses a new `max_score` below the highest existing score (422), so the invariant *score ≤ max_score* can never be broken from either side.

6.6 Transport and browser security

- **CORS allow-list** — only origins listed in `FRONTEND_ORIGIN` receive `Access-Control-Allow-Origin`; preflights for PUT/PATCH/DELETE and JSON bodies are answered correctly. Verified: an unknown origin receives no CORS header.
- **JSON only** — write requests with a non-JSON body get 415; bodies are capped at 100 kB.
- **XSS prevention** — the frontend never passes API data to `innerHTML`. Links are only made clickable after checking the protocol is `http:` / `https:`.
- **Secrets** — configuration lives in `.env` (git-ignored); only `.env.example` is committed. `x-powered-by` is disabled.

7 REST API reference

Base path `/api`. JSON in, JSON out. Resource names are plural nouns; the HTTP method expresses the action.

METHOD	PATH	DESCRIPTION	OK	ERRORS
GET	`/interns?track=&search=&page=&limit=`	Paginated list → `{ data, page, limit, total }`	200	422
GET	`/interns/:id`	Intern + mentor + profile + submissions (JOINs)	200	404, 422
POST	`/interns`	Create (optional `bio`, `linkedin_url`)	201	409, 422
PUT	`/interns/:id`	Replace full record	200	404, 409, 422
PATCH	`/interns/:id`	Partial update (e.g. phone only)	200	404, 409, 422
DELETE	`/interns/:id`	Delete; profile + submissions cascade	204	404, 422
GET	`/mentors`	List with `intern_count`	200	—
GET	`/mentors/:id`	Mentor + their interns	200	404, 422
POST	`/mentors`	Create	201	409, 422
PUT	`/mentors/:id`	Replace	200	404, 409, 422
DELETE	`/mentors/:id`	Delete; interns become unassigned	204	404, 422
GET	`/projects`	List with `submission_count`, `average_score`	200	—
GET	`/projects/:id`	Project + its submissions	200	404, 422
POST	`/projects`	Create	201	409, 422
PUT	`/projects/:id`	Replace (max_score ≥ existing scores)	200	404, 409, 422
DELETE	`/projects/:id`	Delete; submissions cascade	204	404, 422
POST	`/submissions`	Assign project to intern	201	409, 422
PATCH	`/submissions/:id`	Update status / score / github_url	200	404, 422
DELETE	`/submissions/:id`	Unassign	204	404, 422
GET	`/stats`	Totals + top-5 leaderboard	200	—
GET	`/health`	Liveness check	200	—

Error response shape

```
{
  "error": {
    "status": 422,
    "message": "Validation failed",
    "details": [
      { "field": "track",
        "message": "Track must be one of:
frontend, backend, fullstack" }
    ]
  }
}
```

GET /api/stats (seed data)

```
{
  "totalInterns": 10, "totalProjects": 4,
  "pendingReviews": 3, "averageScore": 66.6,
  "leaderboard": [
    { "name": "Ali Raza", "averageScore": 92, ...
},
    { "name": "Priya Sharma", "averageScore": 81,
... },
    ... top 5 by AVG(score), GROUP BY intern
  ]
}
```

Status codes used

200 OK · **201** Created · **204** No Content · **400** malformed JSON · **404** record or route not found · **409** duplicate (23505) · **413** body too large · **415** not JSON · **422** validation / FK / check violation · **500** unexpected (logged only) · **503** database unreachable.

Definitions: `pendingReviews` = submissions with status *submitted* (handed in, awaiting a mentor's review). `averageScore` = `AVG(score)` across all scored submissions.

8 Frontend design and integration

8.1 Pages

PAGE	FEATURES
`index.html` Dashboard	Four stat cards, leaderboard, projects overview, interns table with track filter, debounced search and pagination. Add intern (POST) and edit intern (PUT) in a modal; inline "edit phone" (PATCH); delete with an in-page confirmation dialog.
`intern.html?id=X` Intern detail	Details, mentor, profile (bio, LinkedIn). Assign a project from a list of not-yet-assigned projects (POST). Each submission is an editable card: status, score (with "/ max" hint) and GitHub URL (PATCH), plus Unassign (DELETE).
`projects.html` Projects	Table of projects with assignment count and average score; create / edit in a modal (POST / PUT); delete with confirmation that states how many submissions will be removed.

8.2 JavaScript modules

MODULE	RESPONSIBILITY
`config.js`	`API_BASE_URL`, track/status lists, page size.
`api.js`	The single `request()` function, the `ApiError` class and thin named helpers (`api.createIntern`, …) that all call `request()`.
`ui.js`	Safe DOM builder `el()`, `createView()` state manager, toasts, custom `confirmDialog()`, `withBusy()` button guard, `showFormError()`, `debounce()`, `safeUrl()`.
`dashboard.js` · `intern.js` · `projects.js`	Page logic: loading, rendering, event wiring.

8.3 The central request wrapper

```
export async function request(path, { method = 'GET', body, signal } = {}) {
  const headers = { Accept: 'application/json' };
  const init = { method, headers, signal };
  if (body !== undefined) {
    headers['Content-Type'] = 'application/json';
    init.body = JSON.stringify(body);
  }
  let response;
  try {
    response = await fetch(`${API_BASE_URL}${path}`, init);
  } catch (err) {
    if (err.name === 'AbortError') throw err;          // superseded request
    throw new ApiError(0, 'Cannot reach the server. Check that the API is running and try again.');
  }
  if (response.status === 204) return null;           // DELETE: no body
  const isJson = (response.headers.get('Content-Type') || '').includes('application/json');
  const payload = isJson ? await response.json().catch(() => null) : null;
  if (!response.ok) {
    const error = payload && payload.error;
    throw new ApiError(response.status, error?.message || `Request failed with status ${response.status}`,
error?.details);
  }
  return payload;
}
```

8.4 Loading, empty and error states

Every data region contains three slots — `spinner`, `status` and `content` — managed by `createView()`. A load always shows the spinner, then exactly one of: rendered content, an empty-state message, or an error message with a **Retry** button. The spinner is hidden in a `finally` block. When a newer load starts (e.g. the user keeps typing), the previous request is aborted and its result ignored, so out-of-order responses can never display stale data.

```
async function load(options) {
  if (controller) controller.abort();               // cancel the previous request
  const current = (controller = new AbortController());
  start();                                          // show spinner
  try {
    const data = await options.fetcher(current.signal);
    if (current.signal.aborted) return;
    present(data, options);                         // content or empty state
  } catch (error) {
    if (error.name === 'AbortError') return;
    fail(error, () => load(options));             // error state + Retry
  } finally {
    if (controller === current) { controller = null; finish(); }   // hide spinner
  }
}
```

8.5 Forms and error presentation

- Submit buttons are wrapped in `withBusy()`: disabled and marked `aria-busy` until the request settles, preventing double submissions.

- `showFormError(form, error)` looks for `[data-error-for="<field>"]` slots. For 422 and 409 responses with matching `details`, the message is placed under the input, the input gets `aria-invalid="true"` and receives focus. Anything else (network failure, 500, 404) becomes a toast.
- Forms use `novalidate` so the server is the single source of truth for validation messages.
- Destructive actions use a custom `<dialog>`-based confirmation (Esc and Cancel both decline) — never `window.confirm` or `alert`.

8.6 Safe rendering and responsiveness

- All dynamic content is created with `el(tag, { text, attrs }, children)`, which uses `document.createElement`, `textContent` and `setAttribute`. A bio such as `<img src=x onerror=alert(1)>` is displayed literally.
- The layout uses CSS grid and design tokens. Below 720 px the tables collapse into labelled cards (via `data-label` attributes); below 520 px forms become single-column. No horizontal scrolling at 390 px width.
- After each write the affected regions refresh in place (interns table, stat cards, leaderboard, project panel) — no full page reloads.

9 Application walkthrough

Screenshots captured automatically during the end-to-end browser test run against the seeded database.

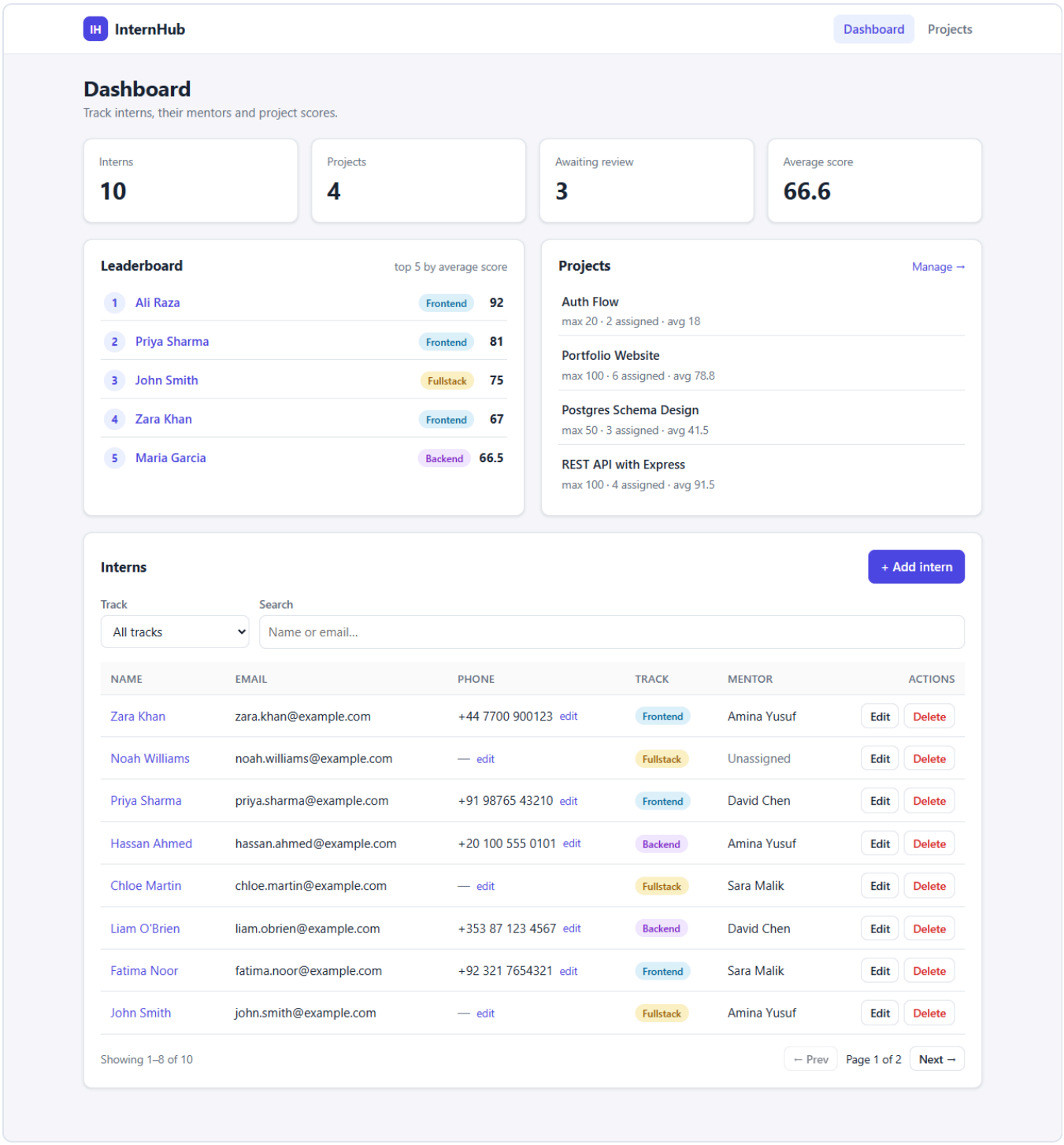


Figure 1 — Dashboard. Stat cards, leaderboard (top 5 by average score), projects overview and the paginated interns table with track filter, search, inline phone edit and row actions. All four datasets were fetched in parallel with `Promise.all`.

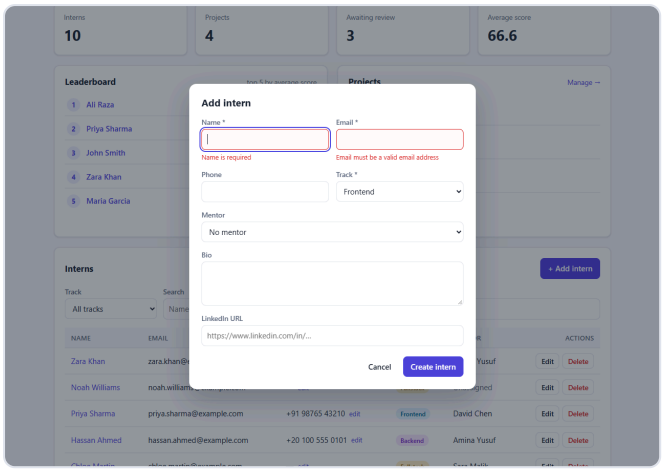


Figure 2 — Server validation (422). Submitting an empty form: the API's per-field messages appear under the matching inputs.

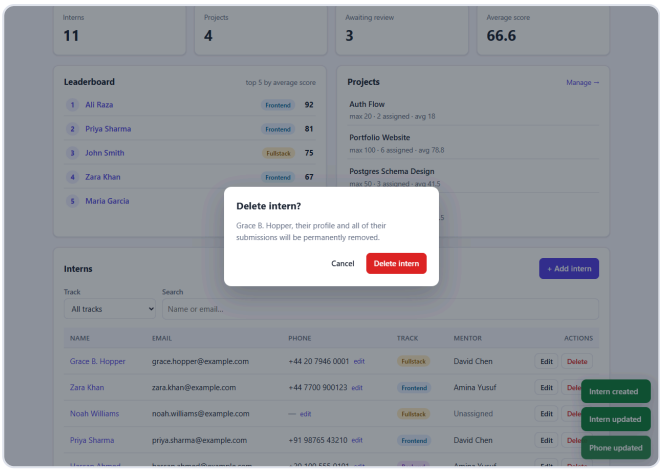


Figure 3 — Custom confirmation. In-page dialog before deleting an intern; success toasts from the preceding create / update / PATCH are visible.

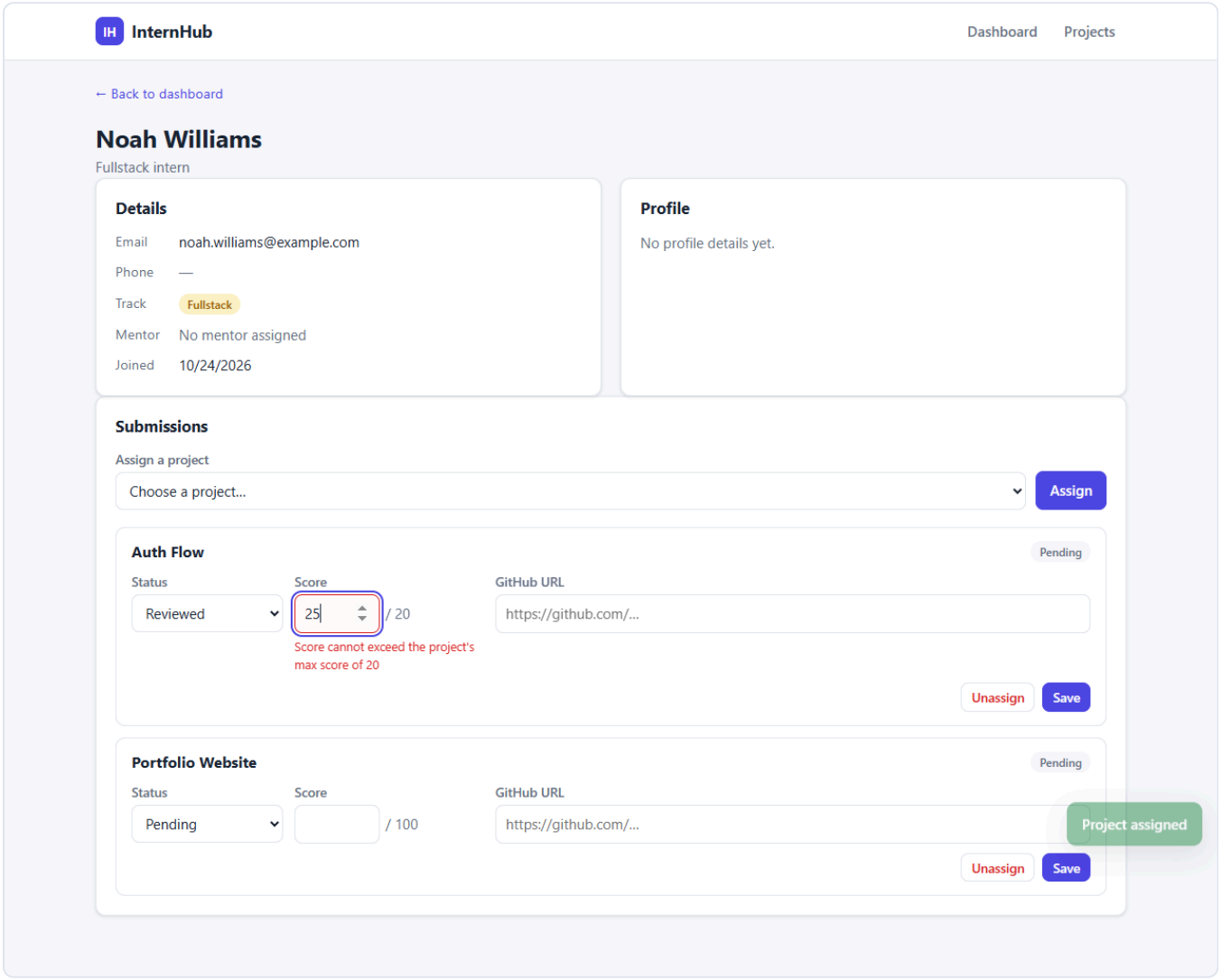


Figure 4 — Intern detail and scoring. A project was just assigned (POST). Entering a score of 25 for a project whose maximum is 20 is rejected by the API with a 422, shown next to the score field.

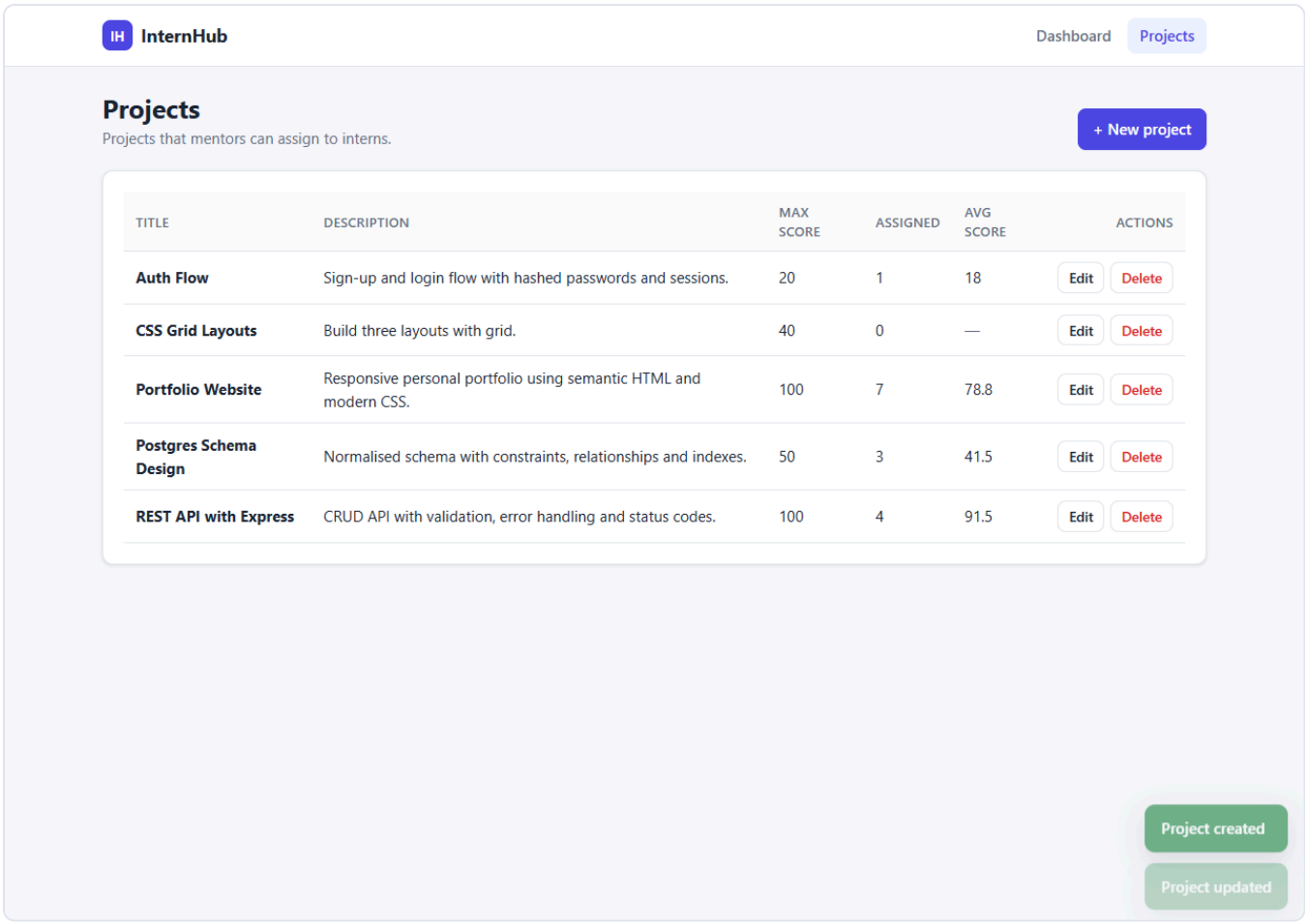


Figure 5 — Projects page. Full CRUD; a new project ("CSS Grid Layouts") was created and its max score edited to 40.

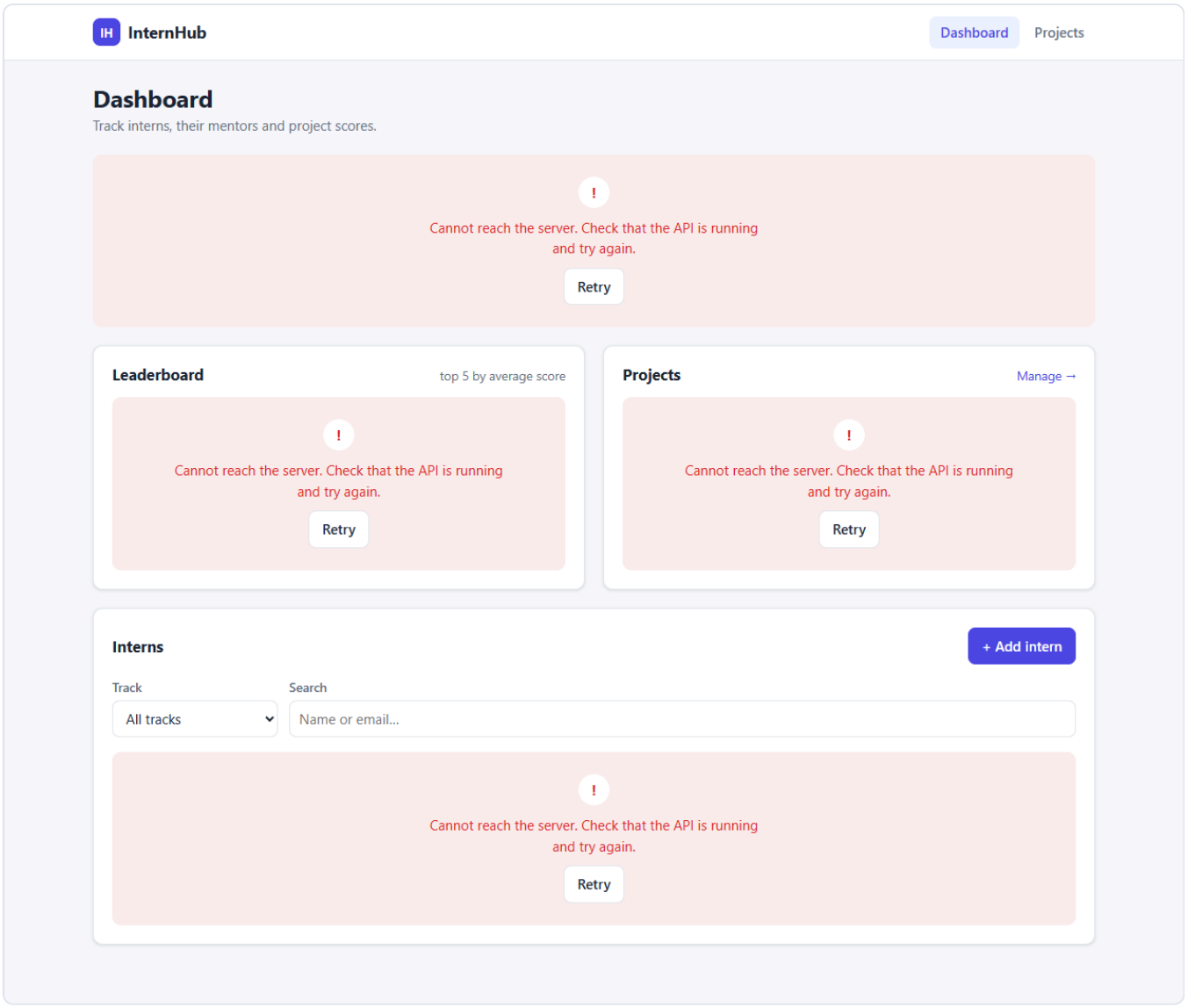


Figure 6 — Backend stopped. Every data region shows a clear error with a Retry button instead of a blank or broken page. After restarting the API, Retry restored all data without reloading.

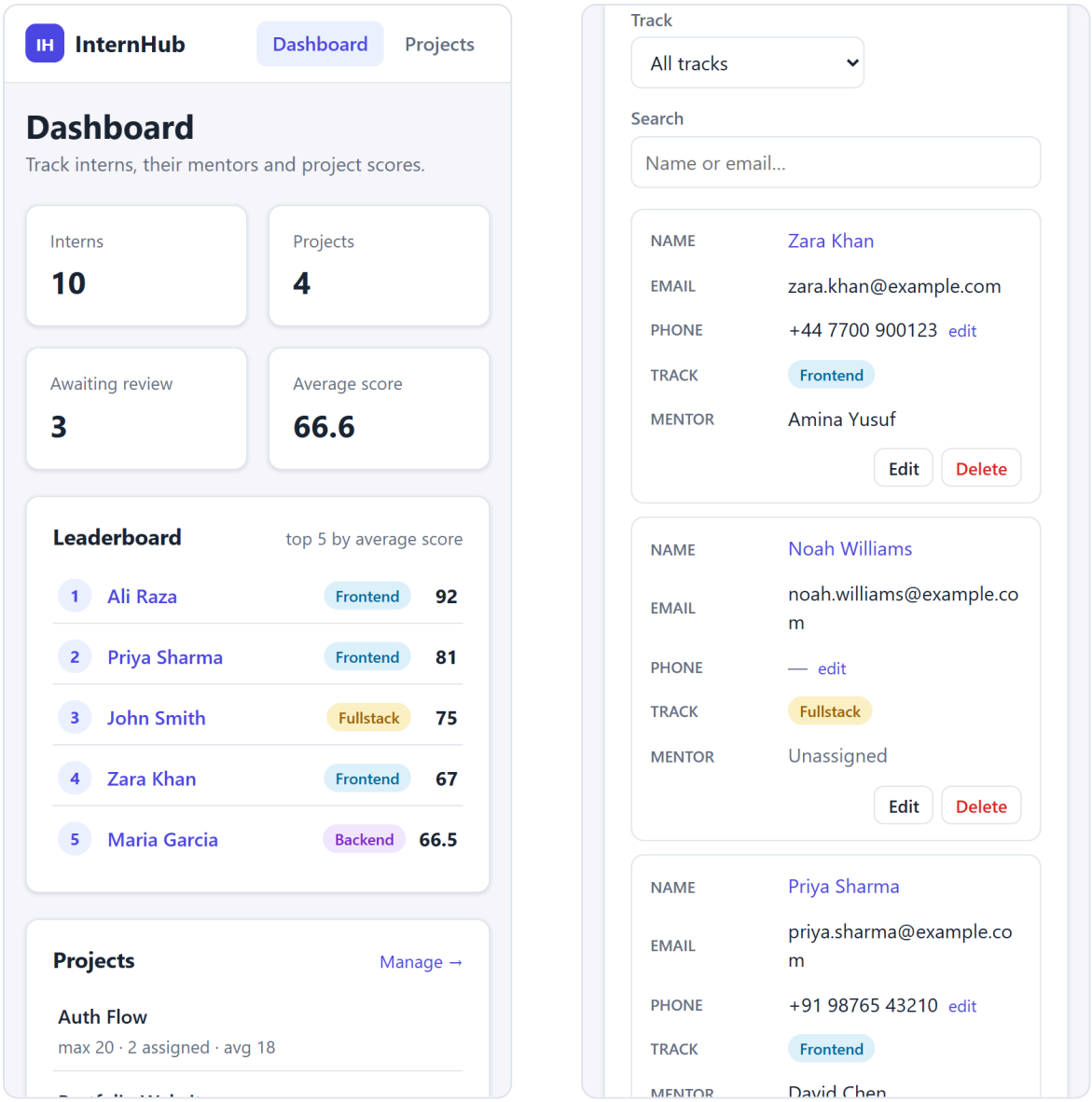


Figure 7 — Mobile (390 px wide). Stat cards reflow to two columns and panels stack (left); the interns table collapses into labelled cards with the same actions (right). No horizontal scrolling.

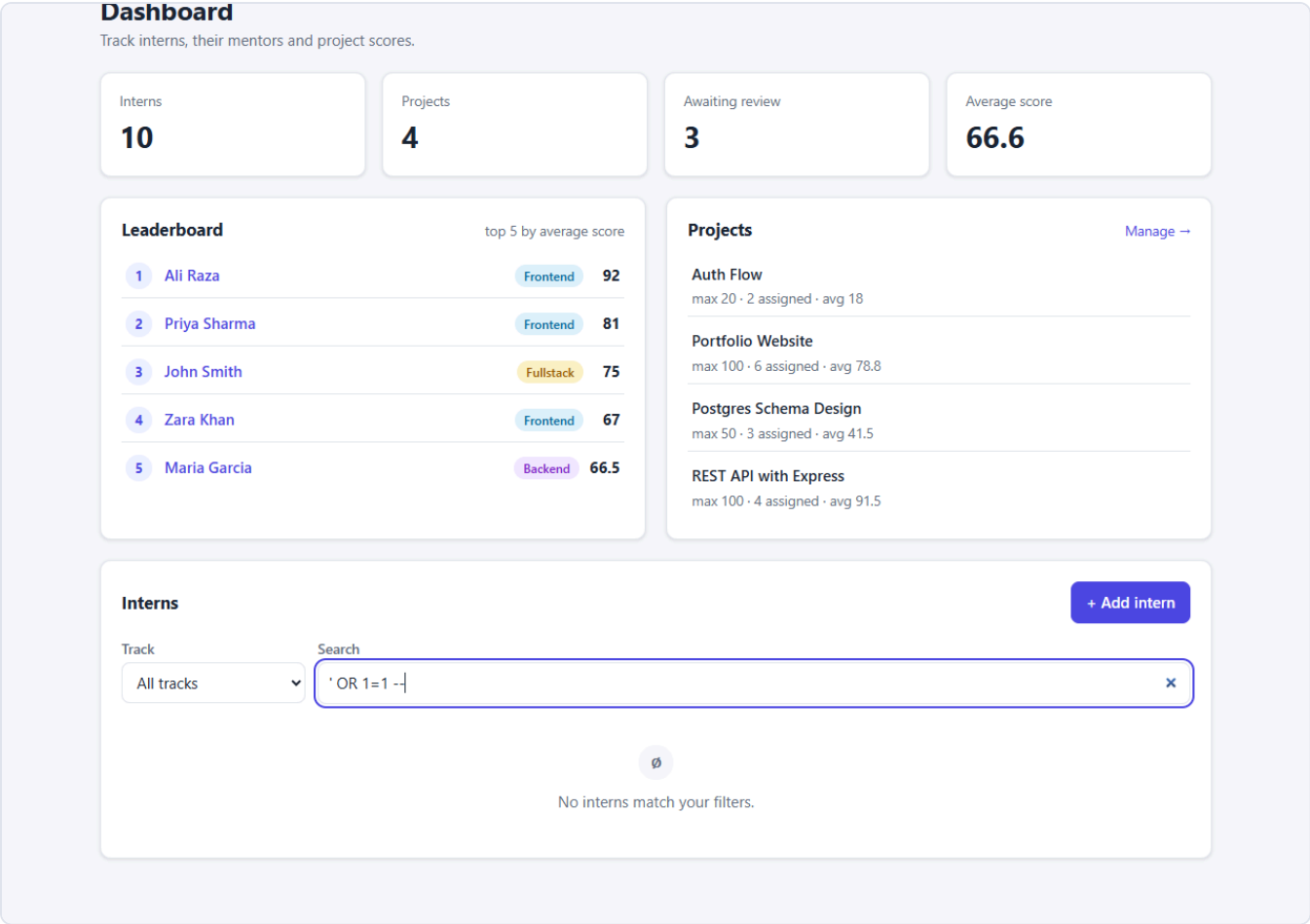


Figure 8 — SQL injection attempt. Searching for `' OR 1=1 --` is treated as plain text: zero matches, empty-state message, no data leaked.

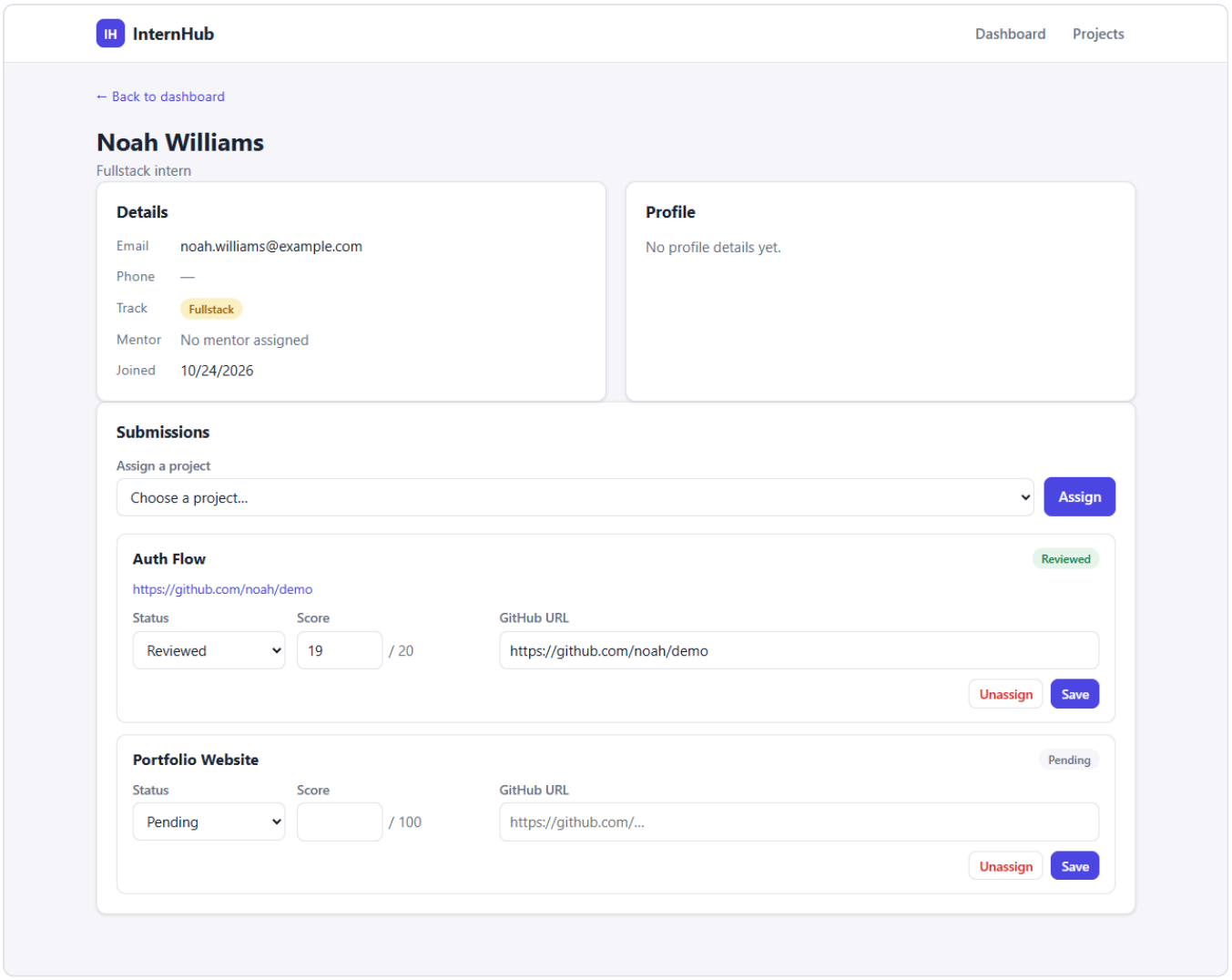


Figure 9 — Saved submission. After a valid score is saved and the page reloaded, the submission shows the *Reviewed* status and persisted score.

10 Testing and verification

Verification was performed against a running PostgreSQL 16 instance, the API on port 3000 and the frontend on port 5500. The database was re-seeded before each run so results are deterministic.

55	49	5	0
API checks (curl)	browser flow checks	outage / recovery checks	failures · console errors

10.1 API tests with curl

AREA	CASES (ALL EXPECTED STATUS CODES RETURNED)	RESULT
Interns — read	List (200); track filter + pagination (200); search (200); SQL-injection search → 0 rows (200); invalid track filter (422); get by id with JOINs (200); nonexistent id (404); non-numeric id (422)	**8 / 8**
Interns — write	Create (201); duplicate email (409); invalid track (422); missing fields + bad email (422); nonexistent mentor FK (422); malformed JSON (400); PUT replace (200); PUT nonexistent (404); PUT invalid (422); PUT duplicate email (409); PATCH phone only (200); PATCH empty body (422); PATCH invalid phone (422); PATCH `name: null` (422); PATCH nonexistent (404); DELETE (204); DELETE again (404)	**17 / 17**
Mentors	List (200); get one (200); create (201); duplicate email (409); PUT (200); DELETE (204); get deleted (404)	**7 / 7**
Projects	List (200); get one (200); create (201); duplicate title (409); max_score 0 (422); PUT (200); lower max_score below an existing score (422); DELETE (204); DELETE again (404)	**9 / 9**
Submissions	Assign (201); assign again (409); missing intern (422); PATCH valid score (200); **score above max (422)**; negative score (422); invalid status (422); nonexistent (404); DELETE (204); DELETE again (404)	**10 / 10**
Stats & misc	Stats (200); unknown route (404); unsupported method on resource (404); non-JSON body (415)	**4 / 4**

10.2 CORS

A preflight `OPTIONS /api/interns/1` with `Access-Control-Request-Method: PATCH` from `http://127.0.0.1:5500` returned `204` with `Access-Control-Allow-Origin: http://127.0.0.1:5500`, the allowed methods and `Content-Type` header. The same preflight from `http://evil.example` received **no** `Access-Control-Allow-Origin` header, so the browser blocks it.

10.3 Browser end-to-end tests (headless Chrome)

FLOW	WHAT WAS VERIFIED	RESULT
Dashboard load	Stat values, 5 leaderboard rows, 4 projects, 8 interns on page 1, "Showing 1–8 of 10", all spinners hidden; page 2 shows the remaining 2	**7 / 7**

FLOW	WHAT WAS VERIFIED	RESULT
Search & filter	`' OR 1=1 --` → empty state; "maria" → exactly Maria Garcia; track = backend → only backend interns	**3 / 3**
Create (POST)	Empty submit → 422 messages under Name and Email; duplicate email → 409 under Email; old errors cleared on resubmit; success toast; new intern first row; stats updated to 11	**6 / 6**
Edit (PUT)	Form pre-filled including profile fields; name and track changed in the table	**3 / 3**
Quick phone edit (PATCH)	Invalid phone → inline 422 message; valid phone saved and displayed	**3 / 3**
XSS	Bio containing `<img src=x onerror=alert(1)>` rendered as text; no `<img>` created; no alert fired	**1 / 1**
Delete	Custom dialog shown with intern's name; Cancel keeps the row; confirm deletes it and stats return to 10	**4 / 4**
Intern detail	Details rendered; assign without project → 422 beside select; assign → new card; score above max → 422 beside score; valid save → Reviewed badge; persisted after reload; unassign; nonexistent intern → error state with Retry	**11 / 11**
Projects CRUD	4 rows; max_score 0 → 422; duplicate title → 409; create; edit max score; delete	**9 / 9**
Responsive	No horizontal overflow at 390 px on dashboard and intern detail	**2 / 2**
Native dialogs	No `window.alert` / `confirm` invoked at any point	✓

10.4 Resilience: backend stopped

SCENARIO	OBSERVED BEHAVIOUR	RESULT
Open dashboard with API down	Each region shows "Cannot reach the server…" with a Retry button; spinners hidden	**2 / 2**
Open projects page with API down	Error state with Retry	**1 / 1**
Restart API, click Retry	Interns table and stats load in place without a page reload	**2 / 2**

11 Setup and running

Prerequisites

Node.js 20+ (LTS) and either a local PostgreSQL 14+ or Docker.

Step 1 — Start PostgreSQL

```
# Option A: Docker (from the internhub/ folder) — creates user/password/db "internhub"
docker compose up -d

# Option B: local PostgreSQL
CREATE USER internhub WITH PASSWORD 'internhub';
CREATE DATABASE internhub OWNER internhub;
```

Step 2 — Configure and initialise the backend

```
cd backend
npm install
cp .env.example .env          # Windows: copy .env.example .env
npm run db:init               # schema + seed in one transaction (safe to re-run)
```

VARIABLE	EXAMPLE	PURPOSE
`PORT`	`3000`	API port
`DATABASE_URL`	`postgres://internhub:internhub@localhost:5432/internhub`	PostgreSQL connection
`FRONTEND_ORIGIN`	`http://127.0.0.1:5500,http://localhost:5500`	CORS allow-list (comma-separated)

Step 3 — Run the API

```
npm run dev                   # nodemon (auto-restart)  — or —  npm start
curl http://localhost:3000/api/health      # → {"status":"ok"}
```

Step 4 — Serve the frontend on port 5500

```
# VS Code: right-click frontend/index.html → "Open with Live Server"
# or any static server:
npx http-server frontend -p 5500 -c-1
# then open http://127.0.0.1:5500
```

> **Note**
> Opening the HTML files directly via `file://` will not work: ES modules and CORS require an HTTP origin. If the API runs elsewhere, edit `API_BASE_URL` in `frontend/js/config.js` and add the frontend's origin to `FRONTEND_ORIGIN`.

Example requests

```
API=http://localhost:3000/api
curl "$API/interns?track=frontend&search=ali&page=1&limit=5"
curl -X POST "$API/interns" -H "Content-Type: application/json" \
  -d '{"name":"Grace Hopper","email":"grace@example.com","track":"backend","mentor_id":2}'
curl -X PATCH "$API/interns/11" -H "Content-Type: application/json" -d '{"phone":"+44 20 7946 0000"}'
curl -X POST "$API/submissions" -H "Content-Type: application/json" -d '{"intern_id":9,"project_id":1}'
curl -X PATCH "$API/submissions/16" -H "Content-Type: application/json" -d
'{"status":"reviewed","score":85}'
curl -X DELETE "$API/interns/11" -i               # 204 No Content
curl "$API/stats"
```

The README contains a curl example for every endpoint and failure case.

12 Design decisions, limitations and future work

Key design decisions

DECISION	RATIONALE
Raw SQL, no ORM	Required by the brief, and it makes every query, JOIN and constraint visible — the learning goal of the database phase.
Express 5	Native async error propagation keeps route handlers short and makes the central error handler the single place where errors become responses.
Map named constraints to fields	Lets the database stay the final authority on uniqueness and references while the UI still shows a precise, field-level message.
Score ≤ max_score enforced in the API	The rule spans two tables, which a plain `CHECK` cannot express and the brief fixed the schema. Row locks on both write paths keep it race-free.
Profile handled with the intern	POST / PUT / PATCH accept `bio` and `linkedin_url` and upsert the 1:1 profile in the same transaction, demonstrating `ON CONFLICT`.
Newest interns first	A freshly created intern appears at the top of page 1, so the user sees immediate confirmation.
Content-Type only on requests with a body	Keeps plain GETs as CORS "simple requests" (no preflight); JSON writes are preflighted and handled.
Server-side validation as source of truth	Forms use `novalidate` so users see the same messages the API enforces, demonstrating the full error round-trip.

Known limitations

- **No authentication or roles.** Anyone who can reach the API can act as a mentor. Out of scope for this training brief.
- **Average score uses raw points.** Projects have different maxima (20 – 100), so `AVG(score)` mixes scales, as the brief specifies. A percentage-based average would be fairer.
- **Mentors have API CRUD but no dedicated UI page**; they are used in the intern form's dropdown.

- **Docker compose file not executed in verification.** Testing used a portable PostgreSQL 16 because Docker was not installed on the build machine; the compose file uses standard settings.
- **Test scripts are not part of the repository.** The curl and browser suites were run during development but are not yet committed as an automated test suite.

Possible next steps

- Authentication (sessions or JWT) with mentor / admin roles and per-mentor data scoping.
- A mentors management page and a per-mentor dashboard.
- Committed automated tests: API integration tests (e.g. `node:test` + a test database) and the browser flow suite in CI.
- Sorting on table columns, percentage-based scoring and a score history / review comments.
- Database migrations instead of drop-and-recreate, for safe schema evolution.

> **Summary**
>
> InternHub meets every requirement in the brief: a constrained, indexed relational schema accessed only through parameterized SQL; consistent, well-mapped API errors; and a framework-free frontend that integrates asynchronously, renders safely, reports errors precisely and never leaves the user facing a blank screen.